Ex.no:12 Date: 03.09.2020

End - End Communication and Transport layer.

a) Implement echo client - server using TCP / UDP socket

b) Implement chat client - server using TCP / UDP Sockets.

Aim:

a) To implement echo client - server using TCP/UDP sockets.

b) To implement chat client - server using TCP/UDP sockets.

Algorithm:

* Implement an echo client - server using socket
* using TCP / UDP sockets for communication between client and server.
* Establish client - server communication through the transport layer.
* Exchange data / message between the client and server.
* Record the input and output obtained during the communication

Part - B chat client - server:

* Implement a chat client - server using socket programming
* use TCP / UDP sockets for communication between the client and server.

TCP:

```
import socket
import threading
#---- SERVER ----
    def start_server():
        server = socket.socket(Socket.AF_INET,
                               socket.sock_STREAM)
        server.bind(('127.0.0.1', 5000))
        server.listen(1)
        print("[SERVER] listening on 127.0.0.1:500")
        conn, addr = server.accept()
        print(f"[server] connected by {addr}")
while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == 'quit'
            print("[SERVER] Received : {data}")
            print("[SERVER] connection closed by client")
            break
        print(f"[SERVER] Received : {data}")
        conn.send(f"Server received : {data}".encode())
    conn.close()
    Server.close()
def start_client():
    client = Socket.socket(Socket.AF_INET, Socket.
                           SOCK_STREAM)
    client.connect(('127.0.0.1', 5000))
```

```
print ("[CLIENT] connected to server")

while true:
    msg = input ("Enter message (or) 'quit' to exit): ")
    client.send(msg.encode())
    if msg.lower() == 'quit':
        break
    response = client.recv(1024).decode()
    print(f"[SERVER RESPONSE] {response}")
client.close()

if __name__ == "__main__":
    server_thread = threading.Thread(target = start_server, daemon = True)
    server_thread.start()
    start_client()
```

Output :

```
[SERVER] listening on 127.0.0.1:5000
[CLIENT] connected to server
[SERVER] connected by ('127.0.0.1', 53455) Enter message (or 'quit' to exit):
hello
[SERVER] Received : hello
[SERVER RESPONSE] Server received : Hi
Enter message (or 'quit' to exit): quit
```

Pg:

[SERVER] connection closed by client.

UDP:

```
import socket
import threading
def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
    while True:
        data, addr = s.recvfrom(1024)
        msg = data.decode()
        if msg.lower() == 'quit':
            print(f"[SERVER] client {addr} ended the connection")
            break
        print(f"[SERVER] received from {addr}: {msg}")
        s.sendto(data, addr)
    s.close()
    print("[SERVER] stopped.")
```

```
if __name__ == "__main__":
    threading.Thread(target=udp_server,
                     daemon=True).start()
udp_client()
```

Output:

```
Enter message (or 'quit' to stop): [SERVER] Running
                on 127.0.0.1:12000
hai
[SERVER] received from ('127.0.0.1', 49670): hi
Echo from server : hi
Enter message : hello
[SERVER] received from ('127.0.0.1', 49670): hello
Echo from server : hello
Enter message : how are you
[SERVER] received from ('127.0.0.1', 49670): how
                                        are you
Echo from server : how are you
Enter a message (or 'quit' to stop): quit
[CLIENT] closing connection .... [SERVER] client
        ('127.0.0.1', 49670) ended the connection.
[CLIENT] stopped [SERVER] stopped
```

Result:

Thus Echo client - server and chat client - server communication using TCP/UDP sockets were successfully implemented